(Don't) Forget About It: Forgetting-Penalized Supervised Learning*

Gaurav Sood†

December 1, 2025

Abstract

When supervised models are retrained on the same task, updates can introduce *model regression*: examples that a previous model classified correctly become misclassified. In many deployments these reversals are costly even if aggregate accuracy improves, because they change which cases are hard, disrupt downstream workflows, and can force organizations to reconfigure human effort. We capture this with a simple economic view: a new model is attractive only when its reduction in expected downstream operating costs (e.g., error handling, staffing, oversight) outweighs the cost of changing behavior on previously well-served examples, which we illustrate with a routing-and-staffing example but which applies more broadly.

Motivated by this view, we propose two lightweight regularizers that discourage within-task regression during standard supervised training. A margin-based *flip penalty* downweights updates that push previously confident examples back toward the decision boundary, and a *Confidence Drop* penalty adds cost whenever an example's loss increases between epochs. Both require only small per-example state and no extra passes over the data. On the Adult Income benchmark, enabling either penalty after a short warm-up reduces cumulative forgetting on a fixed out-of-sample set by 95–98% relative to baseline, while the Confidence Drop penalty keeps validation accuracy within 0.5 percentage points, providing a simple, economically motivated knob for trading flexibility against stability on known-good cases.

*https://github.com/finite-sample/pareto-gd

†Gaurav can be reached at *gsood07@gmail.com*

1 Introduction

Many production machine-learning systems are not trained once and frozen. Instead, they are retrained or fine-tuned as new data arrive, features are added, or objectives evolve. Each update replaces a deployed model with a new one that is often more accurate on average—*but not necessarily more accurate on every example*. Some examples that were previously classified correctly can become incorrect after the update. We call each such correct $\rightarrow$ incorrect transition a *model regression event*.

Model regression is closely related to two better-studied phenomena. First, it is a within-task analogue of catastrophic forgetting in continual learning, where performance on earlier tasks degrades when new tasks are introduced (French, 1999; Kirkpatrick et al., 2017; Zenke et al., 2017; Lopez-Paz and Ranzato, 2017). Second, it is a special case of *prediction churn* and backward incompatibility between model versions, which has been shown to cause friction in downstream systems and user-facing products even when headline metrics improve (Fard et al., 2016; Srivastava et al., 2020; Bahri and Jiang, 2021; Jiang et al., 2022). In both literatures, stability is valued alongside accuracy.

In many applications, regression events are more than a statistical curiosity. Consider human-in-the-loop workflows where models triage cases for specialized teams: fraud analysts, loan officers, medical coders, content moderators. When a new model version performs worse on a particular slice of the population, organizations may need to reallocate staff, invest in retraining, or absorb more exception handling. Section 2 develops a stylized staffing example and shows how even small regressions can erase the benefits of an aggregate accuracy gain, once realignment costs are taken into account.

Standard supervised training, however, optimizes only an average loss over the data. It offers no explicit incentive to preserve correctness on examples that the current model already handles well. In fact, small decreases in performance on some examples can be traded off arbitrarily against improvements elsewhere, as long as the overall loss decreases. This can produce undesirable instability in production pipelines that depend on consistent behavior on a stable set of known-good cases.

In this paper, we develop and study two simple penalties that directly discourage model regression during training on a single supervised task:

1. A **flip penalty** that uses a differentiable margin-based surrogate to penalize examples that were previously classified with a safety margin but subsequently move closer to the decision boundary or flip to the wrong side.

2. A **Confidence Drop** penalty that adds cost whenever an example's loss increases relative to the previous epoch, encouraging per-example loss trajectories that are

approximately monotone non-increasing.

Both penalties are designed to be easy to integrate into existing pipelines. They require (i) a single pass per epoch, (ii) only a small amount of per-example state (a Boolean and, optionally, a margin or scalar loss), and (iii) no changes to model architecture or optimizer. They can be viewed as conservative, example-level regularizers that bias gradient descent away from updates that harm previously well-classified examples.

Our contributions are threefold:

1. We formalize within-task model regression and link it to an economic model of operational costs that makes the value of stability explicit.
2. We introduce two example-level penalties, one flip-based and one loss-based, that operationalize a "do-no-harm" principle for supervised learning without requiring task boundaries, replay buffers, or auxiliary teacher models.
3. We evaluate the penalties on the Adult Income dataset, showing substantial reductions in cumulative forgetting with minimal accuracy costs in a proof-of-concept setting, and we discuss when such penalties are—and are not—appropriate for real-world deployments, including how they interact with alternative mitigations such as routing between model versions.

2 Operational Motivation: Staffing and Economic Costs

To motivate why model regression matters operationally, consider a financial-services firm that uses an ML classifier to triage incoming customer requests and route them to domain experts. Requests are bucketed into categories such as *account inquiries*, *fraud alerts*, *loan applications*, and *technical issues*. Each category is serviced by a dedicated team. The classifier routes straightforward cases to automated handling and flags ambiguous or high-risk cases for human review.

2.1 Economic drivers

Four main forces shape staffing costs in this scenario:

- **Specialization efficiency.** Staff are more productive and make fewer errors when working within their area of expertise.
- **Retraining and transition costs.** Moving staff from one specialty to another requires training and temporarily reduces capacity.

- **Hiring and separation costs.** Changing total headcount entails recruiting, onboarding, or severance expenses.
- **Model accuracy.** Higher model accuracy reduces the volume of cases requiring human intervention, lowering staffing needs across all categories.

Let $C_{\text{staff}}(M)$ denote the ongoing staffing cost when model M is deployed, and let $C_{\text{move}}(M_A, M_B)$ denote the one-time cost of realigning staff when switching from model M_A to M_B. The organization should deploy M_B only if

$$C_{\text{staff}}(M_A) - C_{\text{staff}}(M_B) \; > \; C_{\text{move}}(M_A, M_B). \tag{1}$$

Crucially, C_{move} is sensitive not just to aggregate accuracy, but to *where* the new errors fall. If the update improves accuracy on easy cases but regresses on a small but operationally important subset (e.g., high-value fraud alerts), the realignment cost may dominate.

2.2 Regression events as economic externalities

Let $\mathcal{S}$ denote a fixed set of examples representative of the requests the production system must continue to handle well (e.g., a curated "golden set" or a held-out slice of historical traffic). Define a regression event relative to a baseline model M_A as an example $x \in \mathcal{S}$ such that M_A was correct and M_B is not. Denote the set of such examples by

$$\mathcal{R}(M_A, M_B) \; = \; \{x \in \mathcal{S} : M_A(x) \text{ correct}, \; M_B(x) \text{ incorrect}\},$$

and let $|\mathcal{R}(M_A, M_B)|$ be its size. Suppose each regression event on $\mathcal{S}$ incurs an expected operational cost c_{flip} (e.g., measured in additional analyst-hours or churn risk). Then a simple approximation to the realignment cost is

$$C_{\text{move}}(M_A, M_B) \approx c_{\text{flip}} \cdot |\mathcal{R}(M_A, M_B)|, \tag{2}$$

abstracting away more complex dynamic effects. Combining this with (1) highlights a natural trade-off: a model update that decreases staffing needs by a small amount but introduces many regressions may be economically unattractive.

In this view, model regression is an *externality* of the standard training objective. Cross-entropy loss treats all errors symmetrically and is indifferent to whether a particular example was previously correct. From the perspective of the organization, however, errors on previously-correct examples may be more costly than errors on previously-incorrect ones.

2.3 From economic costs to a regularized objective

Let $\mathcal{L}_{\text{std}}(\theta)$ denote the usual empirical loss of a model parameterized by θ, and let $\mathcal{R}_{\text{flip}}(\theta)$ denote a surrogate measure of expected regression events relative to a reference model (formalized in Section 4). A natural regularized objective is

$$\mathcal{L}_{\text{econ}}(\theta) \;=\; \mathcal{L}_{\text{std}}(\theta) + \lambda_{\text{econ}}\, \mathcal{R}_{\text{flip}}(\theta), \tag{3}$$

with a penalty weight λ_{econ} reflecting the relative cost of regressions vs. aggregate loss.

A simple heuristic for λ_{econ} is to scale it in proportion to c_{flip} and inversely to the marginal benefit of reducing standard loss. For instance, if we write the expected total cost as

$$\text{Cost}(\theta) \approx c_{\text{err}} \cdot \mathcal{L}_{\text{std}}(\theta) + c_{\text{flip}} \cdot \mathcal{R}_{\text{flip}}(\theta)$$

for some effective error cost c_{err}, then minimizing an affine rescaling of this cost leads to (3) with

$$\lambda_{\text{econ}} \propto \frac{c_{\text{flip}}}{c_{\text{err}}}.$$

In practice, we tune the penalty weight empirically, but this framing clarifies how operational considerations should influence that tuning.

3 Background and Related Work

3.1 Example forgetting as a dataset diagnostic

Toneva et al. (2019) formalize *forgetting events* in SGD as correct $\rightarrow$ incorrect transitions for individual training examples and show that forgetting counts are heavy-tailed across examples: some examples are never forgotten, while others flip frequently. The ranking of (un)forgettable examples is surprisingly stable across architectures, and pruning examples that are never forgotten on CIFAR-10 has little effect on accuracy. Follow-up work uses example-level scores based on early training dynamics to identify important examples and prune the training data with minimal loss in performance (Paul et al., 2021). Forgetting statistics have also been used to design reweighting schemes for long-tailed recognition (Yu and Chen, 2022).

In all these cases, forgetting is used *post hoc* as a diagnostic signal for data curation or reweighting. By contrast, we seek to embed a forgetting penalty directly into the training objective to prevent harmful reversals in the first place.

3.2 Catastrophic forgetting and continual learning

Catastrophic forgetting—severe performance loss on previously learned tasks when training on new tasks—has been documented and analyzed in connectionist models for decades (French, 1999). Modern continual-learning research has developed several families of defenses:

Parameter-regularization methods. Elastic Weight Consolidation (EWC) (Kirkpatrick et al., 2017) and Synaptic Intelligence (SI) (Zenke et al., 2017) estimate parameter importance from previous tasks and penalize updates that move important parameters too far. These methods can be understood as task-level analogues of the regularized objective in (3), but they operate on parameters, not per-example outputs.

Rehearsal and replay. Gradient Episodic Memory (GEM) (Lopez-Paz and Ranzato, 2017) and related methods store a subset of past examples and replay them while learning new tasks, effectively anchoring performance on old tasks at the cost of extra memory and computation.

Architectural expansion. Progressive networks (Rusu et al., 2016) and similar approaches allocate new capacity for each task and use lateral connections to avoid interference, trading efficiency for reduced forgetting.

These methods assume task boundaries and aim to preserve *task-level* performance. In contrast, we remain in a single supervised task and focus on output-level stability for individual examples.

3.3 Prediction churn and backward compatibility

In production, successive model versions can disagree on predictions even when trained on essentially the same data and objective. Fard et al. (2016) formalize this as *prediction churn* and propose regularizers that explicitly trade off accuracy against changes in predictions between model versions. Srivastava et al. (2020) provide an empirical analysis of backward compatibility in deployed ML systems, documenting how small distributional changes and training stochasticity can lead to incompatibilities even when overall accuracy improves.

More recent work has proposed techniques to reduce churn while preserving or improving accuracy, including locally adaptive label smoothing (Bahri and Jiang, 2021) and distillation-based approaches that regularize the new model toward the previous

model's logits or soft labels (Jiang et al., 2022). These methods require storing or recomputing predictions from the previous model and typically introduce an auxiliary teacher-student loss.

Our penalties are complementary. Like churn-reduction methods, we care about stability across training iterations, but we operate directly on per-example loss changes along a single training trajectory, without maintaining a separate teacher model or full logits from the previous version.

3.4 Multi-objective and Pareto-optimal training

Multi-task learning can be cast as a multi-objective optimization problem, where each task loss is a separate objective. Lin et al. (2019) introduce Pareto Multi-Task Learning (Pareto MTL), which solves a family of constrained subproblems to approximate the Pareto front of trade-offs between tasks. Navon et al. (2021) go further by training a hypernetwork that maps a preference vector to model parameters, effectively learning to generate the entire Pareto front.

Our setting is different: we have a single task, and we do not enforce strict Pareto constraints. The Confidence Drop penalty discourages per-example loss increases in a soft way by adding a regularizer, and it is often possible—and desirable—to trade off performance on some examples for others. We therefore treat connections to Pareto-style methods as loose intuition rather than literal multi-objective optimization: the penalty encourages moves that resemble per-example Pareto improvements, but does not prohibit violations.

3.5 Knowledge distillation and learning without forgetting

Knowledge distillation (Hinton et al., 2015) trains a student model to match a teacher's soft predictions, often using temperature-scaled logits. In addition to model compression, distillation can be used to keep new models close to previous ones in terms of predictions, and has been explicitly linked to churn-constrained optimization (Jiang et al., 2022). Learning without Forgetting (LwF) (Li and Hoiem, 2016) uses distillation-like losses to preserve performance on old tasks when training on new ones without retaining the original training data.

Our focus is different in two ways. First, we remain within a single task, rather than multi-task or continual learning. Second, we regularize per-example loss changes across epochs of a single model, rather than training a new model to mimic a fixed teacher. This avoids the need to store or update separate teacher models but provides a less global

guarantee.

3.6 Stability and generalization

Theoretical work on *algorithmic stability* shows that learning algorithms whose predictions are insensitive to small perturbations of the training set tend to generalize well (Bousquet and Elisseeff, 2002). Empirically, Toneva et al. (2019) report that examples which are never forgotten during training are often easier and less informative, and that forgetting patterns correlate with generalization performance. Churn-reduction work, in turn, studies prediction stability across retrainings on held-out test data (Bahri and Jiang, 2021; Jiang et al., 2022).

Our penalties operate at training time and are defined on the training data, while our regression metrics are measured out-of-sample. The extent to which training-time stability guarantees out-of-sample stability is thus connected to, but not answered by, the existing stability literature. We return to this question in Section 6.3.

4 Forgetting-Penalized Training

4.1 Problem formulation and forgetting metrics

Let $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^n$ be a binary classification dataset with features $x_i \in \mathbb{R}^d$ and labels $y_i \in \{0, 1\}$. We consider models with parameters θ that produce scalar logits $z_\theta(x) \in \mathbb{R}$ and probabilities $p_\theta(x) = \sigma(z_\theta(x))$, where σ is the logistic sigmoid. The standard empirical risk minimization objective with binary cross-entropy loss is

$$\mathcal{L}_{\text{std}}(\theta) \;=\; \frac{1}{n}\sum_{i=1}^{n} \ell(p_\theta(x_i), y_i), \quad \ell(p, y) = -y \log p - (1 - y) \log(1 - p). \tag{4}$$

We train the model for epochs $t = 0, 1, \ldots, T$, with $\theta^{(t)}$ denoting the parameters at epoch t and $p_i^{(t)} = p_{\theta^{(t)}}(x_i)$. Following Toneva et al. (2019), we define the per-example correctness indicator at epoch t as

$$c_i^{(t)} = \mathbb{I}\big[\hat{y}_i^{(t)} = y_i\big], \quad \hat{y}_i^{(t)} = \mathbb{I}\big[p_i^{(t)} \geq 1/2\big]. \tag{5}$$

A *forgetting event* (or regression event) between epochs $t - 1$ and t is a correct $\to$ incorrect transition:

$$f_i^{(t)} = \mathbb{I}\big[c_i^{(t-1)} = 1 \text{ and } c_i^{(t)} = 0\big]. \tag{6}$$

Let $\mathcal{S}$ be a fixed out-of-sample (OOS) evaluation set (defined precisely in Section 5.1). We train for a total of T epochs and use T_{warm} epochs of warm-up during which the penalties are inactive. To focus on epochs where the penalties can actually act and to avoid early optimization noise, we define *post-warm-up cumulative forgetting* as

$$F^{+}_{\text{cum}} = \sum_{t=T_{\text{warm}}+1}^{T} \sum_{x_i \in \mathcal{S}} f_i^{(t)}. \tag{7}$$

This counts correct $\rightarrow$ incorrect flips on $\mathcal{S}$ only after the warm-up phase, and we use F^{+}_{cum} as our primary forgetting metric in experiments.

4.2 Flip penalty: a margin-based surrogate

We first construct a penalty that targets examples that were previously classified with a comfortable safety margin but become less confident or misclassified at a later epoch. Define the signed margin

$$m_i^{(t)} = (2y_i - 1)\, z_{\theta^{(t)}}(x_i), \tag{8}$$

so that $m_i^{(t)} > 0$ if and only if the example is correctly classified at epoch t, and $|m_i^{(t)}|$ measures confidence in logit space.

We say that example i was *confidently correct* at epoch $t-1$ if $m_i^{(t-1)} \geq \gamma$ for some margin threshold $\gamma \geq 0$. When training epoch t, we would like to penalize cases where such examples are driven below this safety margin. A simple differentiable surrogate for the discrete flip indicator in (6) is

$$R^{(t)}_{\text{flip}}(\theta) = \sum_{i=1}^{n} \mathbb{I}\big[m_i^{(t-1)} \geq \gamma\big] \cdot \max\big(0, \gamma - m_i^{(t)}\big), \tag{9}$$

where the indicator depends only on the previous epoch's margins and is treated as a constant when differentiating with respect to $\theta^{(t)}$.

Intuitively, the penalty is active only for examples that were confidently correct at $t-1$. For such examples it applies a ReLU penalty whenever the current margin $m_i^{(t)}$ falls below the threshold γ. This includes both near-boundary degradations and full misclassifications (which correspond to $m_i^{(t)} < 0$). The hyperparameter γ controls how aggressively we protect previously well-classified examples; in our experiments we set $\gamma = 0$, which penalizes transitions from positive to non-positive margins.

The flip-penalized objective at epoch t is

$$\mathcal{L}^{(t)}_{\text{flip}}(\theta) = \mathcal{L}_{\text{std}}(\theta) + \lambda_{\text{flip}}\, R^{(t)}_{\text{flip}}(\theta), \tag{10}$$

where $\lambda_{\text{flip}} \geq 0$ is a penalty weight. The gradient of $R^{(t)}_{\text{flip}}$ with respect to the current logits is simple: for active examples ($m_i^{(t-1)} \geq \gamma$ and $m_i^{(t)} < \gamma$), it behaves like a linear penalty that pushes $m_i^{(t)}$ back toward γ.

4.3 Confidence Drop penalty

The flip penalty focuses on discrete correctness changes and requires choosing a margin threshold. A more general "do-no-harm" principle is to discourage any increase in per-example loss between epochs, not just flips. For each example i, we track its previous-epoch loss

$$\ell_i^{(t-1)} = \ell\big(p_{\theta^{(t-1)}}(x_i), y_i\big), \tag{11}$$

and define the Confidence Drop penalty as

$$R^{(t)}_{\text{cd}}(\theta) = \sum_{i=1}^{n} \max\big(0, \ell(p_\theta(x_i), y_i) - \ell_i^{(t-1)}\big). \tag{12}$$

The corresponding objective is

$$\mathcal{L}^{(t)}_{\text{cd}}(\theta) = \mathcal{L}_{\text{std}}(\theta) + \lambda_{\text{cd}}\, R^{(t)}_{\text{cd}}(\theta), \tag{13}$$

with penalty weight $\lambda_{\text{cd}} \geq 0$.

This penalty is inactive whenever the current loss on an example is less than or equal to its previous-epoch loss. For examples whose loss has increased, it adds a term proportional to that increase. As a result, gradient updates give extra weight to examples that have recently regressed. This is loosely reminiscent of a Pareto-style preference against loss increases on any coordinate, but here it is implemented as a soft regularizer that can be outweighed by strong gradients from the primary loss; we do not treat each example as a separate objective or enforce a hard Pareto constraint.

The Confidence Drop penalty requires storing a single scalar $\ell_i^{(t-1)}$ per example, updated after each epoch. In large-scale systems, these scalars could be quantized or approximated (e.g., by bucketing examples by loss) to reduce memory usage, but we do not explore such variants here.

4.4 Implementation details

Both penalties are applied after a warm-up phase in which the model is trained with the standard loss (4) alone. The warm-up allows the model to reach a reasonable accuracy level before introducing stability constraints and reduces the impact of early, noisy

Input: Dataset $\mathcal{D}$, penalty type, penalty weight λ, warm-up epochs T_{warm}
Initialize parameters $\theta^{(0)}$ and per-example state arrays (margins or losses);
for $t = 0, 1, 2, \ldots$ *until convergence* **do**
  Compute standard loss $\mathcal{L}_{\text{std}}(\theta^{(t)})$ on current mini-batch;
  if $t > T_{\text{warm}}$ **then**
    Compute penalty $R^{(t)}(\theta^{(t)})$ on the same mini-batch (either $R^{(t)}_{\text{flip}}$ or $R^{(t)}_{\text{cd}}$);
    $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{std}} + \lambda\, R^{(t)}$;
  else
    $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{std}}$;
  end
  Update parameters: $\theta^{(t+1)} \leftarrow \theta^{(t)} - \alpha \nabla_\theta \mathcal{L}_{\text{total}}$;
  Update per-example state (margins or losses) for examples in the mini-batch;
end

Algorithm 1: Training with forgetting-penalized objectives

optimization dynamics on our forgetting metrics.

We implement the penalties in a mini-batch setting as sketched in Algorithm 1. Each example is identified by a stable index, and we maintain arrays of previous-epoch margins $m_i^{(t-1)}$ (for the flip penalty) or losses $\ell_i^{(t-1)}$ (for the Confidence Drop penalty). No additional passes over the data are required: for each mini-batch we read the relevant state, compute the penalty, and update the state with current-epoch values.

In distributed or streaming training regimes, maintaining per-example state may require partitioning the arrays across workers or introducing a simple key-value store. Exploring the engineering trade-offs in such settings is an important direction for future work; here we focus on the single-machine, epoch-based setting.

4.5 When forgetting penalties may be inappropriate

The penalties above encode a conservative preference for per-example stability. This is desirable only when past predictions are a reliable proxy for desired behavior. There are important scenarios where this is not the case:

- **Label noise and dataset cleaning.** If earlier training phases or baseline models were fit to noisy or systematically biased labels, enforcing consistency with their predictions can entrench those errors and resist correction when better labels are introduced.
- **Major policy or guideline shifts.** In regulated or rapidly evolving domains (e.g., content moderation policies, clinical guidelines), updates are sometimes intended

to overturn previous model behavior. In such cases, penalizing departures from past predictions is counterproductive.

- **Fairness interventions.** When re-training models to reduce disparate error rates across groups, a do-no-harm constraint relative to the previous model can preserve historical biases. Stability should then be evaluated relative to a fairness-aware reference, not the previous model.

In practice, we recommend applying forgetting penalties only to a vetted subset of examples where preserving behavior is clearly desirable (e.g., a golden set of critical cases), and evaluating subgroup metrics alongside aggregate accuracy and forgetting to detect unintended side effects.

5 Experiments

5.1 Dataset and model

We evaluate the proposed penalties in a proof-of-concept study on the Adult Income dataset (Kohavi, 1996). The task is to predict whether an individual's annual income exceeds $50K based on 14 demographic and employment features after standard preprocessing. Following common practice, we use the original training/test split, yielding 32,561 training examples and 16,281 test examples.

Our base model is a multi-layer perceptron (MLP) with two fully connected hidden layers of 64 ReLU units each and a scalar logit output. We train with the Adam optimizer (learning rate 0.01) for 50 epochs, using binary cross-entropy loss. Penalties are activated after 10 warm-up epochs ($T_{\text{warm}} = 10$).

5.2 Baselines and penalized variants

We compare three training strategies:

- **Baseline**: standard cross-entropy training with no forgetting penalty.
- **Flip penalty**: cross-entropy plus the flip penalty objective (10) with $\gamma = 0$.
- **Confidence Drop**: cross-entropy plus the Confidence Drop objective (13).

For both penalties we use a fixed penalty weight and warm-up schedule tuned coarsely on the validation set. Systematic sweeps over λ and T_{warm} are left for future work.

5.3 Evaluation protocol and metrics

To approximate a production setup, we construct a fixed OOS evaluation set $\mathcal{S}$ from the original validation data. Our evaluation protocol proceeds as follows:

1. Train a baseline "production" model on the training data using standard cross-entropy for 50 epochs (with the same learning-rate schedule).
2. For each training strategy (baseline, flip penalty, Confidence Drop), track correctness indicators $c_i^{(t)}$ on $\mathcal{S}$ for all epochs t.
3. For each strategy, compute the post-warm-up cumulative forgetting F_{cum}^{+} as in (7), summing correct $\to$ incorrect transitions on $\mathcal{S}$ for epochs $t = T_{\text{warm}} + 1, \ldots, 50$.

For each method, we report:

- **Post-warm-up cumulative forgetting** F_{cum}^{+} on $\mathcal{S}$.
- **Final training accuracy** on the training set after 50 epochs.
- **Final validation accuracy** on a held-out validation split.

Focusing on post-warm-up epochs avoids conflating two different phenomena: early-epoch oscillations that are largely an optimization artifact, and later regressions that are more relevant for deployment. It also ensures that all methods are evaluated over the same horizon in which penalties are active for the penalized strategies.

5.4 Results

Table 1 summarizes the results. Both penalties dramatically reduce post-warm-up cumulative forgetting relative to baseline.

Table 1. Adult Income results. Post-warm-up cumulative forgetting is the total number of correct $\to$ incorrect transitions on a fixed out-of-sample evaluation set over epochs 11–50.

Method	Post-warm-up Cumulative Forgetting	Final Train Acc	Final Val Acc
Baseline	566	0.794	0.788
Flip penalty	12	0.759	0.760
Confidence Drop	29	0.786	0.783

Three observations stand out:

- **Substantial forgetting reduction.** Both penalties reduce post-warm-up cumulative forgetting on $\mathcal{S}$ by more than an order of magnitude relative to the baseline (from 566 to 12 and 29, respectively).
- **Trade-off between stability and accuracy.** The flip penalty yields the lowest forgetting count but at a noticeable cost to accuracy (about a 2.8 percentage point drop in validation accuracy relative to baseline). This reflects its more aggressive protection of previously correct examples.
- **Favorable trade-off for Confidence Drop.** The Confidence Drop penalty cuts cumulative forgetting by roughly 95% relative to baseline while keeping validation accuracy within about 0.5 percentage points of the baseline model. This makes it an attractive default for deployments where small accuracy losses are acceptable in exchange for much greater stability.

Our cumulative metric F^{+}_{cum} still conflates different dynamic patterns—for example, an example that flips once late in training is counted the same as one that flips multiple times after warm-up. However, by restricting to post-warm-up epochs, we reduce the influence of early oscillations that are unlikely to matter for deployment.

These experiments are deliberately modest in scope: a single tabular dataset and architecture, with no hyperparameter sweeps or alternative churn-reduction baselines. They are best interpreted as evidence that even very simple per-example penalties can substantially reduce model regression in standard supervised settings.

6 Discussion

6.1 Practical implications

Our empirical results suggest that forgetting-penalized objectives can be a useful tool in productionized supervised learning pipelines, especially when:

- There is a stable set of high-value examples (e.g., a golden set, critical test cases, or regulatory benchmarks) on which regression is particularly costly.
- Human-in-the-loop workflows must be staffed around the model's error profile. Large shifts in which cases are difficult may require costly staff retraining or reassignment.

- Downstream systems or users are sensitive to changes in predictions (e.g., customer-facing risk scores or content labels) and prefer gradual, predictable evolution over abrupt shifts.

Compared to more heavyweight approaches like replay buffers or distillation against a separate teacher, the penalties we propose are cheap to implement: they only require storing a small amount of per-example state and do not alter the model architecture. In settings where per-example indices are already maintained (e.g., for logging or curriculum learning), this can be an incremental change.

6.2 Routing between model versions as an alternative mitigation

The penalties we study are one way to align training with a preference for stability, but they are not the only way to manage regression risk. A complementary deployment strategy is to *route* traffic between model versions rather than selecting a single global winner.

Suppose we have two models, M_A (the incumbent) and M_B (an updated candidate), and a feature space of contexts $z(x)$ that define segments or regions of operation (e.g., product lines, geographies, customer types). If we can estimate segment-level performance for both models, we can train a router $g(z)$ that chooses which model to apply for a given request:

$$M_{\text{router}}(x) = \begin{cases} M_A(x), & \text{if } g(z(x)) = A, \\ M_B(x), & \text{if } g(z(x)) = B. \end{cases}$$

Offline, one can estimate for each segment the expected error and regression cost under M_A and M_B, and learn g to minimize an explicit operational objective (e.g., expected staffing plus churn costs), subject to constraints such as capacity limits or fairness requirements. This is essentially a constrained two-expert mixture-of-experts scheme where the router is restricted to choose among existing models rather than learning new experts.

Routing has several advantages. It avoids forcing a single global model to dominate everywhere, allows M_B to be deployed only in regions where it is clearly better, and permits gradual rollout: as evidence accumulates that M_B dominates M_A in more regions, the router can shift traffic accordingly. It also integrates naturally with our economic framing: in segments where M_B offers small accuracy gains but large regression-induced costs, the router can stick with M_A without requiring that M_B be globally penalized during training.

However, routing also introduces complexity. Two (or more) models must be main-

tained and monitored; the router itself may need to be retrained as distributions shift; and segmenting by context can raise fairness concerns if different populations are systematically exposed to different models. In addition, when training budgets are tight, maintaining multiple strong models may not be feasible. In such settings, forgetting-penalized objectives for a single model, combined with careful evaluation on critical segments, remain attractive due to their simplicity.

6.3 Train vs. out-of-sample stability

Our penalties are defined on the training data, but our forgetting metric F^{+}_{cum} is measured on a fixed OOS evaluation set. A natural question is whether increasing stability on the training set tends to increase stability out-of-sample.

Theoretical work on algorithmic stability shows that if a learning algorithm's predictions do not change much when a single training example is modified or removed, then its generalization error can be tightly controlled (Bousquet and Elisseeff, 2002). This notion of stability is about *sensitivity to training perturbations*, rather than about per-example forgetting events, but it suggests that certain kinds of stability may correlate with better-behaved generalization.

Empirically, Toneva et al. (2019) find that examples that are frequently forgotten during training often correspond to noisy or hard-to-learn patterns, while examples that are never forgotten tend to be easier and less informative, and forgetting patterns correlate with generalization performance. Work on prediction churn (Bahri and Jiang, 2021; Jiang et al., 2022) directly measures stability across retrainings on held-out sets, demonstrating that training-time regularizers can reduce churn on test data without hurting accuracy.

Our results show that training-time penalties defined on the training data can substantially reduce forgetting on a disjoint OOS set in the Adult setting. However, we do not claim a general theorem connecting training-time forgetting penalties to out-of-sample stability. Understanding when and how per-example training stability propagates to prediction stability on future data is an interesting open problem that sits between the algorithmic stability, example forgetting, and churn literatures.

6.4 Limitations and future work

Several limitations of the present study merit emphasis:

- **Single dataset and architecture.** We evaluate only on Adult Income with a small MLP. The behavior of the penalties for image, text, and structured-sequence tasks,

and for larger models, is unknown and should be examined empirically.

- **No hyperparameter sweeps.** We fix the penalty weight and warm-up schedule based on coarse tuning. A more thorough exploration of the accuracy–stability trade-off as a function of λ and T_{warm} would provide a clearer picture.
- **Per-example state and training regime assumptions.** Our implementation assumes epoch-based training with stable example indices. Adapting the approach to streaming, sharded, or federated settings will likely require approximate or aggregated variants of the penalties.
- **No subgroup or fairness analysis.** We did not examine whether forgetting or stability varies across demographic subgroups in Adult, nor how the penalties interact with fairness objectives. This is an important gap given the potential for stability constraints to preserve historical biases.
- **Cumulative forgetting metric.** Even when restricted to post-warm-up epochs, the cumulative forgetting metric F^{+}_{cum} does not distinguish between examples that flip once near the end of training and those that oscillate several times. For deployment, late-stage flips may matter more. Future work should consider metrics that emphasize late-training stability (e.g., net regressions between the best validation checkpoint and the final model).

Future work could extend this study along several dimensions: (i) adding image and text benchmarks and more diverse architectures; (ii) comparing against distillation- and parameter-distance-based churn baselines; (iii) analyzing subgroup-specific forgetting and accuracy; (iv) incorporating routes between model versions as in Section 6.2 into the training and deployment pipeline; and (v) exploring variants that anchor loss trajectories to a fixed baseline model rather than the immediately previous epoch.

6.5 When stability is a bug, not a feature

As discussed in Section 4.5, stability is not always desirable. If the goal of a model update is to correct historical biases, incorporate new labeling guidelines, or adapt to significant distribution shift, a forgetting penalty can slow or partially prevent the intended change. In such settings, the relevant reference is ground truth or updated policy, not the previous model's predictions.

We therefore view forgetting-penalized objectives as tools for *graceful updates* in regimes where the target concept and policies are relatively stable and where frequent

regression events on known-good examples are costly. They should be combined with careful monitoring and validation, not applied automatically to every training run.

7 Conclusion

We have introduced two simple forgetting-penalized objectives for supervised learning on a single task. The flip penalty uses a margin-based surrogate to discourage confident examples from crossing back toward the decision boundary, and the Confidence Drop penalty discourages per-example loss increases between epochs. Both require only a small amount of per-example state and no architectural changes.

In a proof-of-concept experiment on the Adult Income dataset, these penalties reduced post-warm-up cumulative forgetting events on a fixed out-of-sample set by 95–98% relative to baseline, with the loss-based penalty maintaining validation accuracy within 0.5 percentage points of standard training. This suggests that substantial gains in stability can be achieved with modest accuracy costs in at least some settings.

We also argued that the penalty weight can be informed by an explicit economic model of operational costs, and we highlighted scenarios where stability constraints may be harmful, such as label corrections, major policy shifts, or fairness interventions. As machine-learning systems become more tightly integrated into business-critical and regulated processes, we expect techniques for managing the externalities of model updates—including prediction churn, backward compatibility, and model regression—to play a growing role in responsible deployment. Forgetting-penalized objectives are one small step in that direction.

References

Bahri, D. and Jiang, H. (2021). Locally adaptive label smoothing improves predictive churn. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 532–542. PMLR.

Bousquet, O. and Elisseeff, A. (2002). Stability and generalization. *Journal of machine learning research*, 2(Mar):499–526.

Fard, M. M., Cormier, Q., Canini, K., and Gupta, M. R. (2016). Launch and iterate: Reducing prediction churn. In *Advances in Neural Information Processing Systems*, volume 29.

French, R. M. (1999). Catastrophic forgetting in connectionist networks. *Trends in Cognitive Sciences*, 3(4):128–135.

Hinton, G., Vinyals, O., and Dean, J. (2015). Distilling the knowledge in a neural network. *arXiv preprint arXiv:1503.02531*.

Jiang, H., Narasimhan, H., Bahri, D., Cotter, A., and Rostamizadeh, A. (2022). Churn reduction via distillation. In *International Conference on Learning Representations (ICLR)*.

Kirkpatrick, J., Pascanu, R., Rabinowitz, N., Veness, J., Desjardins, G., Rusu, A. A., Milan, K., Quan, J., Ramalho, T., Grabska-Barwińska, A., et al. (2017). Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526.

Kohavi, R. (1996). Scaling up the accuracy of naive-Bayes classifiers: A decision-tree hybrid. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 202–207. AAAI Press.

Li, Z. and Hoiem, D. (2016). Learning without forgetting. In *Computer Vision – ECCV 2016*, volume 9908 of *Lecture Notes in Computer Science*, pages 614–629. Springer.

Lin, X., Zhen, H.-L., Li, Z., Zhang, Q., and Kwong, S. (2019). Pareto multi-task learning. In *Advances in Neural Information Processing Systems*, volume 32, pages 12037–12047.

Lopez-Paz, D. and Ranzato, M. (2017). Gradient episodic memory for continual learning. In *Advances in Neural Information Processing Systems*, volume 30.

Navon, A., Shamsian, A., Fetaya, E., and Chechik, G. (2021). Learning the Pareto front with hypernetworks. In *International Conference on Learning Representations (ICLR)*.

Paul, M., Ganguli, S., and Dziugaite, G. K. (2021). Deep learning on a data diet: Finding important examples early in training. In *Advances in Neural Information Processing Systems*, volume 34, pages 20596–20607.

Rusu, A. A., Rabinowitz, N. C., Desjardins, G., Soyer, H., Kirkpatrick, J., Kavukcuoglu, K., Pascanu, R., and Hadsell, R. (2016). Progressive neural networks. *arXiv preprint arXiv:1606.04671*.

Srivastava, M., Nushi, B., Kamar, E., Shah, S., and Horvitz, E. (2020). An empirical analysis of backward compatibility in machine learning systems. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 3272–3280.

Toneva, M., Sordoni, A., des Combes, R. T., Trischler, A., Bengio, Y., and Gordon, G. J. (2019). An empirical study of example forgetting during deep neural network learning. In *International Conference on Learning Representations (ICLR)*.

Yu, H. and Chen, J. (2022). Class-balanced loss based on example forgetting for long-tailed recognition. *SSRN Electronic Journal*.

Zenke, F., Poole, B., and Ganguli, S. (2017). Continual learning through synaptic intelligence. In *Proceedings of the 34th International Conference on Machine Learning*, volume 70 of *Proceedings of Machine Learning Research*, pages 3987–3995.